

Overview

 In this course, you will explore in detail the processes of GUI testing in Squish, focusing on its architecture, command-line execution, tagging, and report generation features, enhancing your proficiency in distributed testing environments and various testing scenarios.

What will I learn ?

1. Understand the fundamental elements and their processes used to perform testing with Squish.
2. How to run tests from the command line.
3. How to execute tests by using tags.
4. How to generate reports from test results.

[squishserver | Squish 9.2](#)

[squishrunner | Squish 9.2](#)

Squish Architecture

There are two fundamental elements used to perform testing with Squish: The Application Under Test (AUT) and the test script for testing the AUT.

When testing with Squish, these elements are executed in two separate processes. Separating them in two processes ensures Squish will not crash if the AUT does. If the AUT does crash, the test script will simply fail and log an error message. It also allows for simpler remote testing of AUTs running on multiple platforms or embedded devices.

SQUISHDIR refers to the directory of your Squish installation

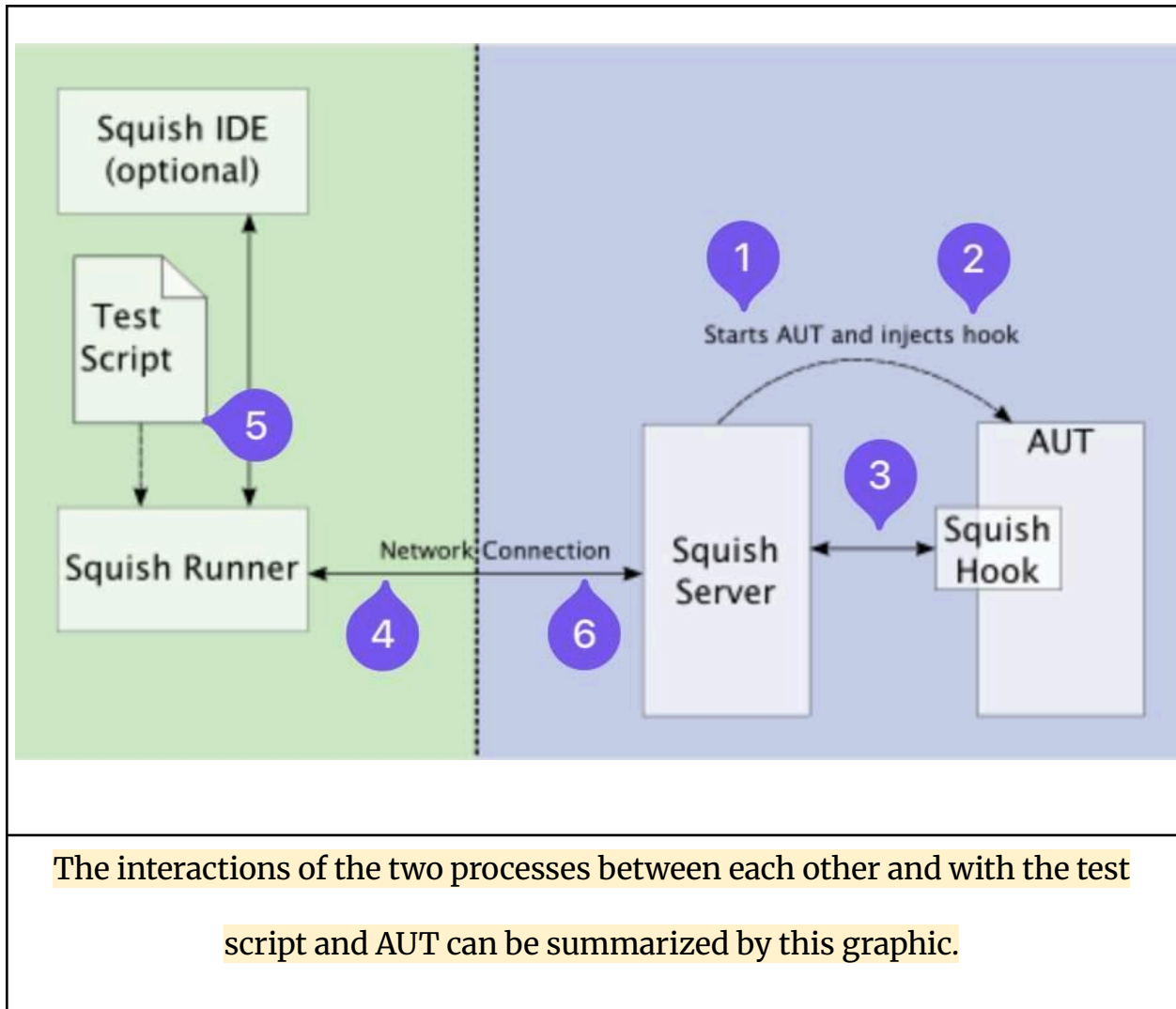
The first process, called [squishserver](#) (SQUISHDIR/bin/squishserver), is a small server run by Squish on the same machine as the AUT. It handles the communication with the AUT and the test script by executing the following steps:

1. Starts the instrumented AUT/Attaches to the AUT on the device
2. Hooks into the AUT (The hook is a small library that makes the AUT's live running objects accessible and allows communication with squishserver)
3. Executes the commands of the test script on the AUT

The second process is called [squishrunner](#) (SQUISHDIR/bin/squishrunner). This tool interprets the test script:

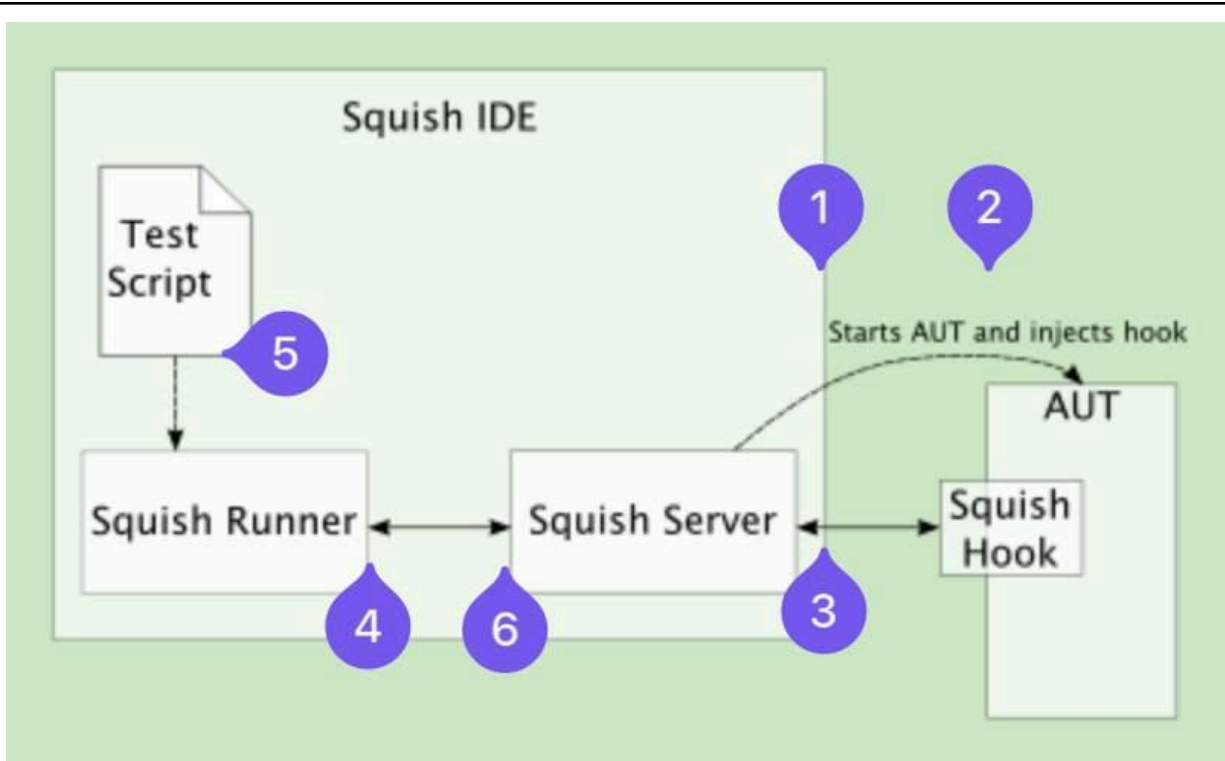
4. Connects to squishserver.

5. Interprets the test script.
6. Sends the commands of the script to squishserver.



The interactions of the two processes between each other and with the test script and AUT can be summarized by this graphic.

When using the Squish IDE, squishserver is started and stopped automatically, and the test results are displayed in the Squish IDE's Test Results view. Squish handles everything behind the scenes when the Squish IDE is used, so from the test engineer's perspective, the separation is not noticeable.



The interactions between the squishrunner and squishserver **within** the **Squish IDE** can be summarized by this graphic.

Understanding how to use squishrunner and squishserver is particularly useful when preparing and executing automatic batch testing, doing testing on different machines, or when using an IDE other than the Squish IDE.

By default, squishserver only accepts connections from squishrunner running on the same machine. To make it accept connections from other hosts, those hosts must be specified in **SQUISHDIR/etc/squishserverrc**.

Below is an example of how to modify this file to allow a machine at the IP address **192.168.0.10** access to the squishserver.

None

```
ALLOWED_HOSTS=192.168.1.10
```

Running Tests from the Command Line

The first step when running tests from the command line is to start a server. This can be done from the command line by simply running [squishserver](#). If the squishserver is started without any command line options, it will listen to connections on **port 4322** from the **local host (127.0.0.1)**.

Several options can be specified when starting squishserver:

--port	Specifies which port to listen on.
--host	Specifies the IP address to listen on.
--verbose	Makes squishserver produce more output than usual. The extra output is designed to be helpful for investigating problems such as remote server connection problems or incorrect Qt library path settings.
--daemon	<i>(Only available on Unix-like platforms)</i> Used to run the squishserver in the background. This means that the server will detach itself from the controlling terminal and stop printing anything to standard output or to standard error.
--logfile	Used to write all log messages as well as any redirected AUT output to the given file path.
--disable-presence	Disables SSDP service discovery. By default, the squishserver informs that it exists on the LAN using this protocol.

The following snippet illustrates the start of squishserver from the command line:

Shell

```
SQUISHDIR\bin>squishserver --port 4322 --host localhost --verbose
```

```
>> Debug: Using server configuration file  
'SQUISH_USER_SETTINGS_DIR/server.ini'
```

```
>> Info: Starting server of Squish version 7.2.0.  
  
>> Debug: Read configuration file  
'C:/Squish/Squish_for_Qt65x_7_2_0/etc/squishserverrc'  
  
>> Info: Listening on 127.0.0.1:4322  
  
>> Info: Announcing server presence via SSDP
```

The server can then be stopped anytime by pressing **Ctrl + C**.

Once the server is started, the [squishrunner](#) is launched. The most common options used when executing tests with squishrunner are:

--host	Used when squishrunner must connect to a squishserver running on a different machine.
--port	Used when squishserver listens to a port other than the default 4322.
--local	Makes squishrunner automatically start and stop a local squishserver process as needed during the playback of a test suite.
--testsuite	The path to the test suite to execute. By default, all the tests in the test suite are executed.
--testcase	Used to specify which test to execute in the test suite.

The following snippet shows how to execute a specific test case locally. Since the option **--local** is used, squishrunner will automatically start a local squishserver process so the host and port are not set:

```
None  
SQUISHDIR\bin>squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4
```

```
--testcase tst_product --testcase tst_sum --local
```

To execute testcases in a random order, we can use:

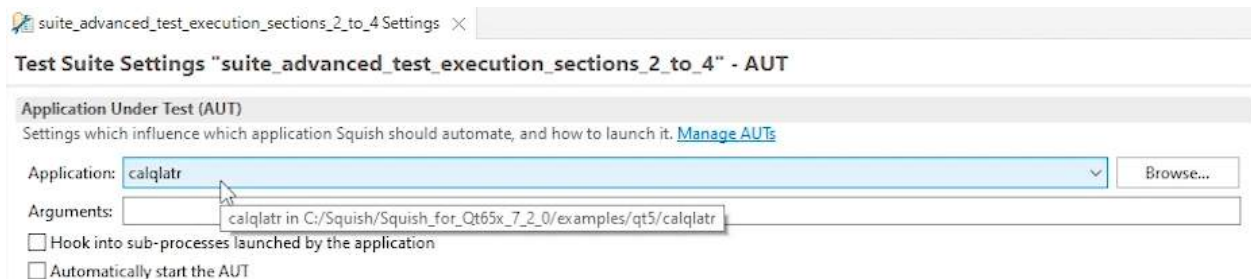
--random	The used sequence number is printed in the report as log message and can be given as a parameter to --random to reproduce a specific order.
----------	---

None

```
SQUISHDIR\bin>squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--random 2877322191 --local
```

Steps

STEP 1=The Squish Test suite



For these tests, we use the Qt Quick Calculator example. The application can be found in the examples subdirectory in the Squish installation. The test suite has 3 test cases. We can run them from the command line, so let's close the Squish IDE.

STEP 2=Starting the squish server


```
Command Prompt - squishserver --host localhost --port 4322 --verbose

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishserver --help
Squish Server 7.2.0
Usage: squishserver [--quiet] [--verbose]
                  [--configfile <configfile>]
                  [--host <address>] [--port <port>]
                  [--local] [--listenfile <listenfile>]
                  [--daemon [--logfile <logfile>]]
                  [--disable-presence]
squishserver --config <action> <options>
squishserver --stop [--host <address>] [--port <port>]
squishserver --version

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishserver --host localhost --port 4322 --verbose
11:31:39:889 Debug: Using server configuration file 'C:\Users\pijaouen\AppData\Roaming\froglogic\Squish\ver1/server.ini'
squishserver: Failed to register with Windows firewall
11:31:39:957 Info: Starting server of Squish version 7.2.0.
11:31:39:961 Debug: Read configuration file 'C:/Squish/Squish_for_Qt65x_7_2_0/etc/squishserverrc'
11:31:39:965 Info: Listening on 127.0.0.1:4322
11:31:39:966 Info: Listening on ws://127.0.0.1:54563?Ugafj_58UtQnyrkEvgOp
11:31:39:966 Info: Announcing server presence via SSDP
```

With a command prompt open, we can navigate to the Squish bin folder within our installation directory. From here, we can start by launching the Squish server and looking at what options are available to us when running **squishserver** by using the **--help** option. We launched a server using the **--port --host** and **--verbose** options. To stop the server, you can press CTRL + C when active in the command prompt window.

STEP 03 = Starting The Squish Runner and Running a Test Suite

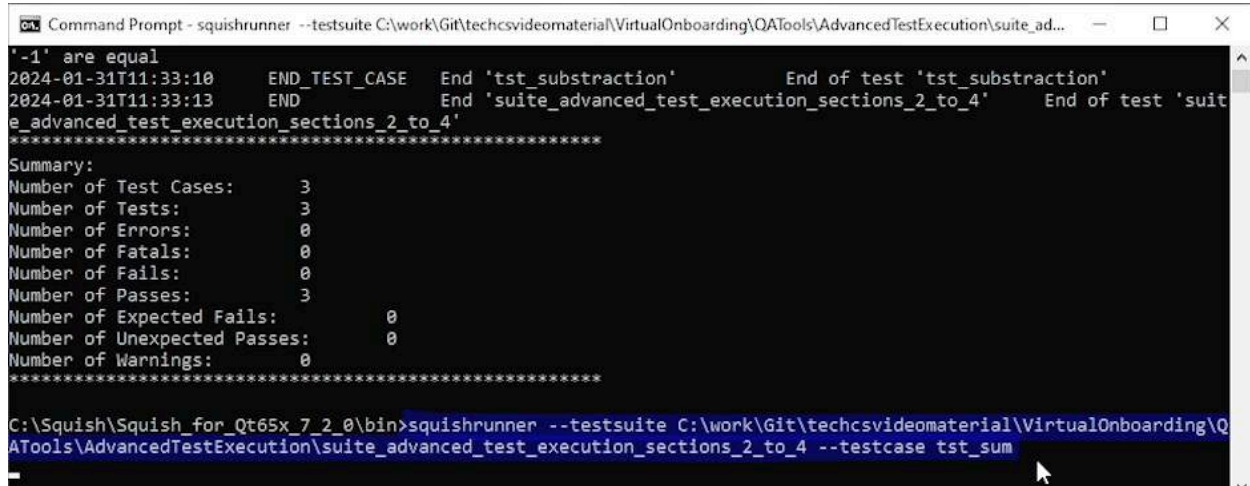
```
Command Prompt - squishrunner --host localhost --port 4322 --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\Adv...

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --host localhost --port 4322 --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4
2024-01-31T11:32:42 START Start 'suite_advanced_test_execution_sections_2_to_4' Test 'suite_advanced_test_execution_sections_2_to_4' started
2024-01-31T11:32:44 START_TEST_CASE Start 'tst_product' Test 'tst_product' started (tst_product)
2024-01-31T11:32:49 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\tst_product\test.py:12: Comparison '2' and '2' are equal
2024-01-31T11:32:49 END_TEST_CASE End 'tst_product' End of test 'tst_product'
2024-01-31T11:32:54 START_TEST_CASE Start 'tst_sum' Test 'tst_sum' started (tst_sum)
2024-01-31T11:33:00 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\tst_sum\test.py:12: Comparison '11' and '11' are equal
2024-01-31T11:33:00 END_TEST_CASE End 'tst_sum' End of test 'tst_sum'
2024-01-31T11:33:05 START_TEST_CASE Start 'tst_substraction' Test 'tst_substraction' started (tst_substraction)
```

Now we can open a second command prompt to configure and run the squishrunner. From squishrunner, we can execute an entire test suite by specifying details like the host and port of our Squish server and the path of the test suite we want to execute (--testsuite).

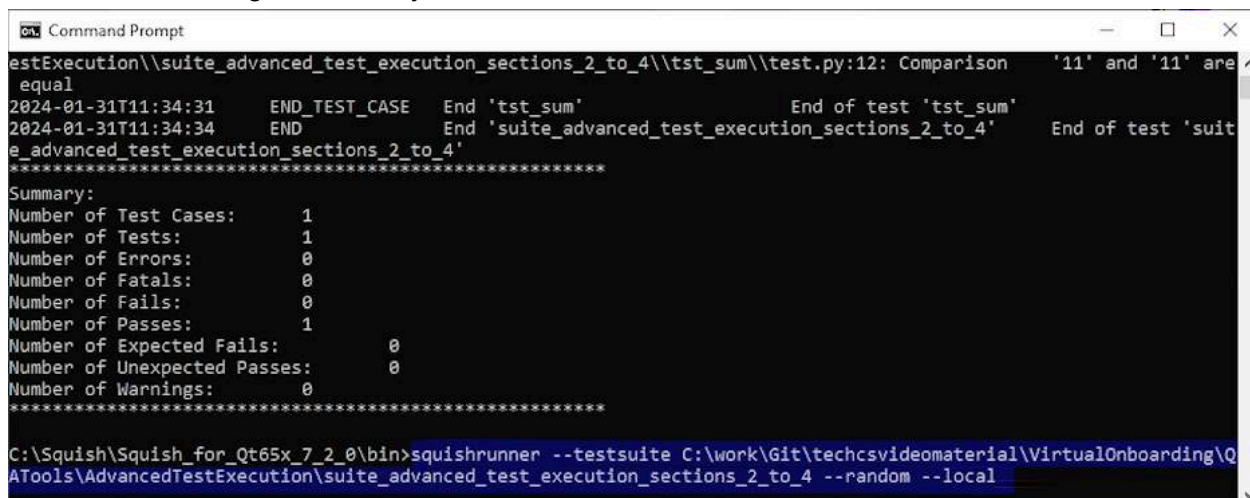
STEP 04 = Executing a Specific Test Case



```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_ad...
'-1' are equal
2024-01-31T11:33:10      END_TEST_CASE      End 'tst_substraction'      End of test 'tst_substraction'
2024-01-31T11:33:13      END      End 'suite_advanced_test_execution_sections_2_to_4'      End of test 'suite_advanced_test_execution_sections_2_to_4'
*****
Summary:
Number of Test Cases:      3
Number of Tests:           3
Number of Errors:          0
Number of FataIs:          0
Number of Fails:           0
Number of Passes:          3
Number of Expected Fails:  0
Number of Unexpected Passes: 0
Number of Warnings:        0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --testcase tst_sum
```

We can execute a specific test case using the `--testcase` option. Note, when executing this in the video we did not specify a host or port for squishrunner or squishserver as we are using the default values.

STEP 05 = Running Test Locally and Randomization

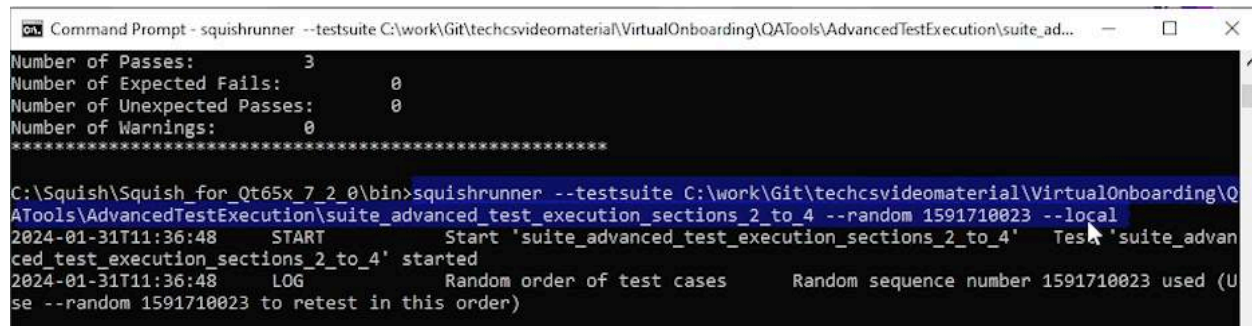


```
Command Prompt
estExecution\\suite_advanced_test_execution_sections_2_to_4\\tst_sum\\test.py:12: Comparison '11' and '11' are equal
2024-01-31T11:34:31      END_TEST_CASE      End 'tst_sum'      End of test 'tst_sum'
2024-01-31T11:34:34      END      End 'suite_advanced_test_execution_sections_2_to_4'      End of test 'suite_advanced_test_execution_sections_2_to_4'
*****
Summary:
Number of Test Cases:      1
Number of Tests:           1
Number of Errors:          0
Number of FataIs:          0
Number of Fails:           0
Number of Passes:          1
Number of Expected Fails:  0
Number of Unexpected Passes: 0
Number of Warnings:        0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --random --local
```

A more convenient approach to running a test locally is using the `--local` option when executing a squishrunner command. In the example, we also used the

--**random** option to randomize the execution order of the test cases. This approach means we can vary and simplify the testing by randomizing the order of test cases and avoid the steps of launching a Squish server as it is handled for us.

STEP 06 = Re-running random tests



```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_ad...
Number of Passes:      3
Number of Expected Fails: 0
Number of Unexpected Passes: 0
Number of Warnings:    0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --random 1591710023 --local
2024-01-31T11:36:48 START Start 'suite_advanced_test_execution_sections_2_to_4' Test 'suite_advanced_test_execution_sections_2_to_4' started
2024-01-31T11:36:48 LOG Random order of test cases Random sequence number 1591710023 used (Use --random 1591710023 to retest in this order)
```

When running a random test, a random number sequence is displayed, allowing us to rerun the tests again in that specific order. The number sequence can be added to --**random** option to rerun the tests in the same order as previously demonstrated.

Tags

The [squishrunner's](#) option **--testcase** can be used to select the test cases to execute. Another option is to use tags.

Tags can be assigned to a test case in the Squish IDE from the Test Management view in the Edit Tags & Description pane. Here, each test case can be assigned one or several tags. Once the tags are set, the **--tags** option can be used to execute Test Cases matching the tag when launching squishrunner.

The following snippet shows how to locally execute all the test cases with the tag **ForDataDriven**:

None

```
SQUISHDIR\bin>squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--tags ForDataDriven --local
```

The following snippet shows how to locally execute all the test cases with the tag

Regression or Smoke:

None

```
SQUISHDIR\bin> squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--tags Regression,Smoke --local
```

The following snippet shows how to locally execute all the test cases with the tag

ForDataDriven and Smoke:

None

```
SQUISHDIR\bin> squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--tags ForDataDriven --tags Smoke --local
```

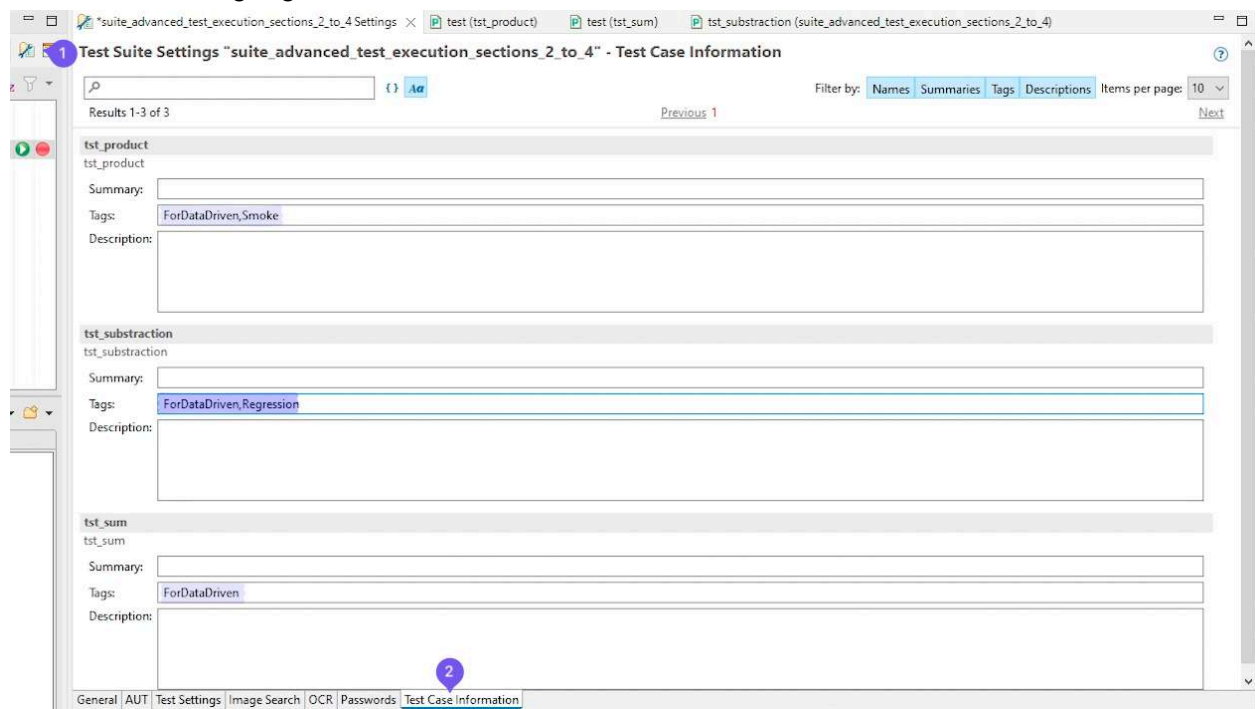
The following snippet shows how to locally execute all the test cases **without** the tag **Regression**. For this, the symbol ~ is used:

None

```
SQUISHDIR\bin>squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--tags ~Regression --local
```

STEPS

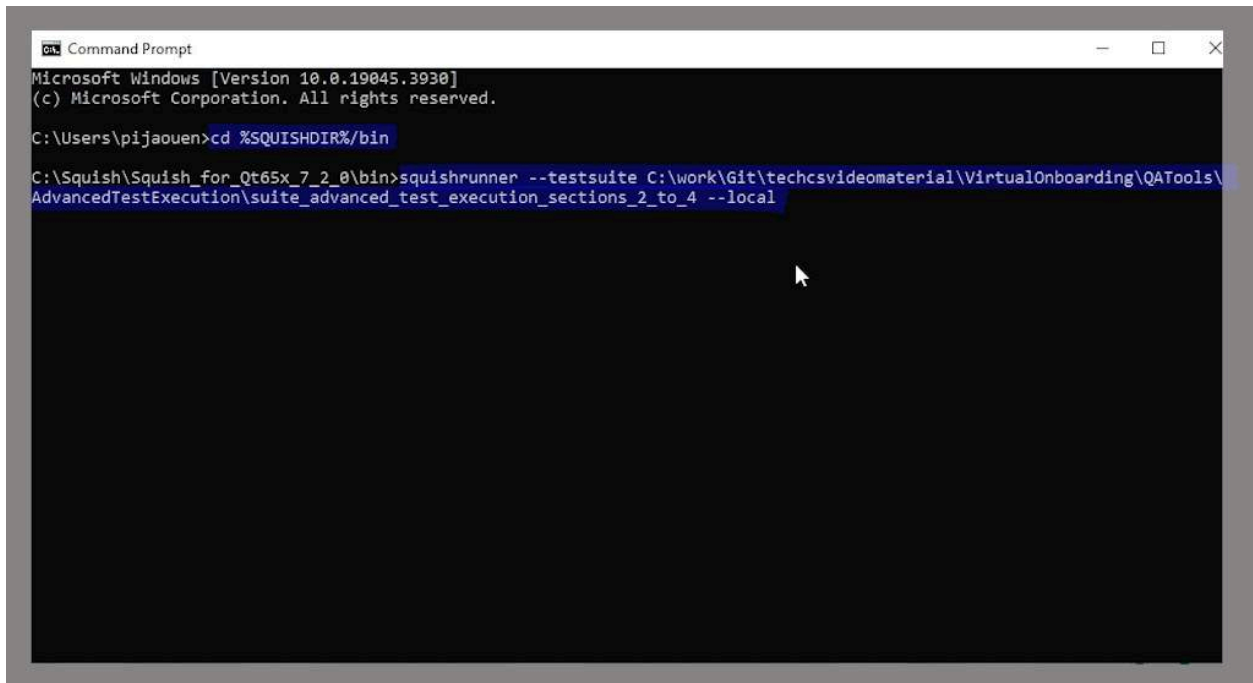
STEP 01 = Adding tags to test cases



We start by looking at how to add tags to our test cases using the Squish IDE. In the Test Case Information pane within the Test Suite Settings, we tag each test case. For example, we added the 'ForDataDriven' tag, and discuss how these could benefit from data-driven approaches. Then for the **tst_product** we will add the

tag **Smoke** and for **tst_subtraction** the tag **Regression**. Each test can have more than one tag. Save the Test Suite Settings and exit the Squish IDE.

STEP 02 = Launching squishrunner Locally



```
Command Prompt
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>cd %SQUISHDIR%/bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\
AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --local
```

Open a new command prompt, navigate to the Squish bin folder in our installation directory and use the **--local** option at the end of the squishrunner command. This starts the squishserver process automatically, streamlining our test execution. Before executing the command, we can specify what tags to look for and execute the desired test cases.

STEP 03 = Executing Tests with Specific Tags


```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_te...
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --tags ForDataDriven --local

2024-01-31T11:50:43 START Start 'suite_advanced_test_execution_sections_2_to_4' Test 'suite_advanced_test_execution_sections_2_to_4' started
2024-01-31T11:50:47 START_TEST_CASE Start 'tst_product' Test 'tst_product' started (tst_product)
11:50:51:487 Wrapper: calqlatr.exe[45852]: Failed to load wrapper extension squishqtprintsupport: The specified module could not be found.
11:50:51:488 Wrapper: calqlatr.exe[45852]: Failed to load wrapper extension QtQuickWebEngineExt: The specified module could not be found.
11:50:51:488 Wrapper: calqlatr.exe[45852]: Failed to load wrapper extension SquishQtQuickWidgetsExtension: The specified module could not be found.
11:50:51:662 AUT stderr: Property QWheelEvent::QPointingDevice has unknown type const.
11:50:51:662 AUT stderr: Property QInputDevice::capabilities has unknown type QFlags<QInputDevice::Capability>.
11:50:51:663 AUT stderr: Property QInputDevice::systemId has unknown type long long.
11:50:51:663 AUT stderr: Property QPointingDeviceUniqueId::numericId has unknown type long long.
2024-01-31T11:50:53 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\tst_product\test.py:12: Comparison '2' and '2' are equal
2024-01-31T11:50:53 END_TEST_CASE End 'tst_product' End of test 'tst_product'
2024-01-31T11:50:57 START_TEST_CASE Start 'tst_sum' Test 'tst_sum' started (tst_sum)
```

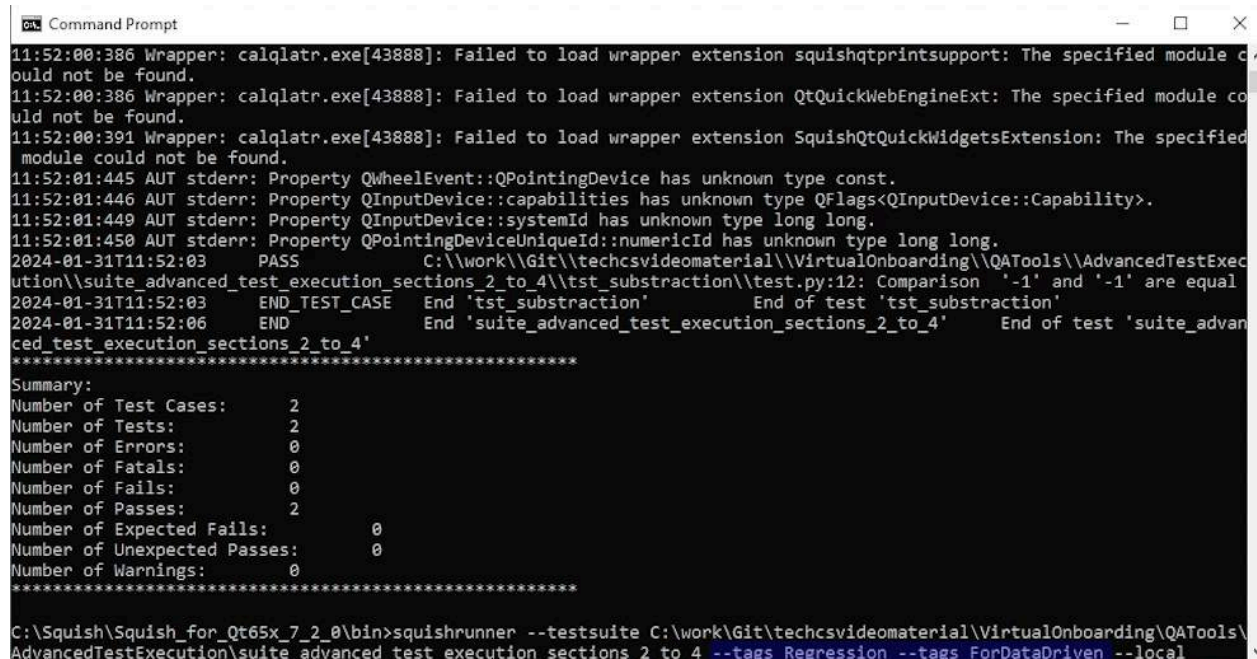
Using the `--tags` option, we can execute test cases with specific tags. We run all test cases tagged 'ForDataDriven'. This approach allows us to execute tests that meet our current testing requirements selectively.

STEP 04 = Running Tests with Multiple Tags

```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_te...
ould not be found.
11:51:10:316 Wrapper: calqlatr.exe[15504]: Failed to load wrapper extension QtQuickWebEngineExt: The specified module could not be found.
11:51:10:318 Wrapper: calqlatr.exe[15504]: Failed to load wrapper extension SquishQtQuickWidgetsExtension: The specified module could not be found.
11:51:11:253 AUT stderr: Property QWheelEvent::QPointingDevice has unknown type const.
11:51:11:253 AUT stderr: Property QInputDevice::capabilities has unknown type QFlags<QInputDevice::Capability>.
11:51:11:255 AUT stderr: Property QInputDevice::systemId has unknown type long long.
11:51:11:256 AUT stderr: Property QPointingDeviceUniqueId::numericId has unknown type long long.
2024-01-31T11:51:12 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\tst_substraction\test.py:12: Comparison '-1' and '-1' are equal
2024-01-31T11:51:12 END_TEST_CASE End 'tst_substraction' End of test 'tst_substraction'
2024-01-31T11:51:15 END End 'suite_advanced_test_execution_sections_2_to_4' End of test 'suite_advanced_test_execution_sections_2_to_4'
*****
Summary:
Number of Test Cases: 3
Number of Tests: 3
Number of Errors: 0
Number of Fatals: 0
Number of Fails: 0
Number of Passes: 3
Number of Expected Fails: 0
Number of Unexpected Passes: 0
Number of Warnings: 0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --tags Regression,Smoke --local
```

By listing tags separated by commas, we execute tests that are marked with any of the listed tags. This method is helpful for running a broader range of test cases in one command.

STEP 05 = Executing Tests with Combined Tags



```
11:52:00:386 Wrapper: calqlatr.exe[43888]: Failed to load wrapper extension squishqtprintsupport: The specified module c
ould not be found.
11:52:00:386 Wrapper: calqlatr.exe[43888]: Failed to load wrapper extension QtQuickWebEngineExt: The specified module co
uld not be found.
11:52:00:391 Wrapper: calqlatr.exe[43888]: Failed to load wrapper extension SquishQtQuickWidgetsExtension: The specified
module could not be found.
11:52:01:445 AUT stderr: Property QWheelEvent::QPointingDevice has unknown type const.
11:52:01:446 AUT stderr: Property QInputDevice::capabilities has unknown type QFlags<QInputDevice::Capability>.
11:52:01:449 AUT stderr: Property QInputDevice::systemId has unknown type long long.
11:52:01:450 AUT stderr: Property QPointingDeviceUniqueId::numericId has unknown type long long.
2024-01-31T11:52:03 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExec
ution\suite_advanced_test_execution_sections_2_to_4\tst_subtraction\test.py:12: Comparison '-1' and '-1' are equal
2024-01-31T11:52:03 END_TEST_CASE End 'tst_subtraction' End of test 'tst_subtraction'
2024-01-31T11:52:06 END End 'suite_advanced_test_execution_sections_2_to_4' End of test 'suite_advan
ced_test_execution_sections_2_to_4'
*****
Summary:
Number of Test Cases: 2
Number of Tests: 2
Number of Errors: 0
Number of FataIs: 0
Number of Fails: 0
Number of Passes: 2
Number of Expected Fails: 0
Number of Unexpected Passes: 0
Number of Warnings: 0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\
AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --tags Regression --tags ForDataDriven --local
```

We can use the `--tags` option twice in the command line to run tests that **must** have both specified tags. This approach narrows down our test execution to very specific criteria.

STEP 06 = Excluding Tests with Certain Tags


```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_te...
ould not be found.
11:52:55:708 Wrapper: calqlatr.exe[47692]: Failed to load wrapper extension QtQuickWebEngineExt: The specified module co
uld not be found.
11:52:55:711 Wrapper: calqlatr.exe[47692]: Failed to load wrapper extension SquishQtQuickWidgetsExtension: The specified
module could not be found.
11:52:56:729 AUT stderr: Property QWheelEvent::QPointingDevice has unknown type const.
11:52:56:730 AUT stderr: Property QInputDevice::capabilities has unknown type QFlags<QInputDevice::Capability>.
11:52:56:733 AUT stderr: Property QInputDevice::systemId has unknown type long long.
11:52:56:735 AUT stderr: Property QPointingDeviceUniqueId::numericId has unknown type long long.
2024-01-31T11:52:58 PASS C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExec
ution\suite_advanced_test_execution_sections_2_to_4\tst_substraction\test.py:12: Comparison '-1' and '-1' are equal
2024-01-31T11:52:58 END_TEST_CASE End 'tst_substraction' End of test 'tst_substraction'
2024-01-31T11:53:01 END End 'suite_advanced_test_execution_sections_2_to_4' End of test 'suite_advan
ced_test_execution_sections_2_to_4'
*****
Summary:
Number of Test Cases: 1
Number of Tests: 1
Number of Errors: 0
Number of FataIs: 0
Number of Fails: 0
Number of Passes: 1
Number of Expected Fails: 0
Number of Unexpected Passes: 0
Number of Warnings: 0
*****
C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\
AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --tags ~Regression --local
```

To exclude specific tests, use the tilde(~) notation in the **--tags** option. By specifying **~Regression**, for instance, we run all tests that don't have the 'Regression' tag. This method is particularly useful for avoiding specific sets of tests during execution.

Report Creation

By default, [squishrunner](#) only logs a summary of the Test Result, but it is possible to configure the generation of the results report when starting squishrunner:

--reportgen

Sets the report generator type and its output directory. It is used with directory-based report generators (json, html, xml3).

Below is an example of using **--reportgen** to generate a static HTML page showing the results, which are being stored in JSON format. The destination for the report is

TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4/report.

None

```
SQUISHDIR\bin> squishrunner --testsuite  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4  
--reportgen html,  
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4/report
```

--resultdir

Determines the location to save the test report and failed screenshot files, if any. It is only used in combination with file-based report generators (stdout, xmljunit, xml2).

The following snippet shows an example of using **--resultdir** to generate an Excel report in a specific directory.

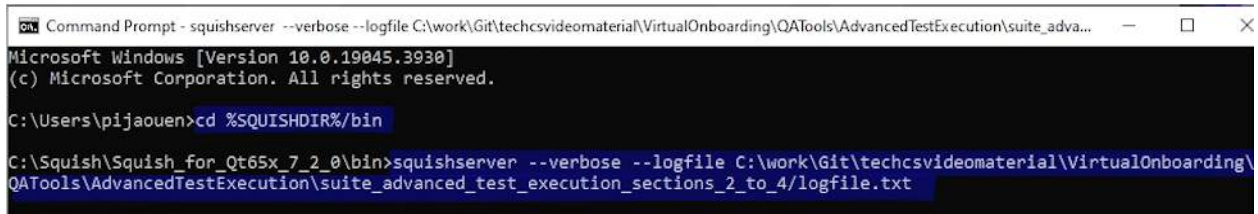
None

```
SQUISHDIR\bin> squishrunner --testsuite
```

```
TESTSUITEFOLDER\suite_advanced_test_execution_sections_2_to_4
--resultdir TESTSUITEFOLDER\suite_advanced_test_execution
_sections_2_to_4/results --reportgen xls
```

STEPS

STEP 01 = Launching a Server with a Logfile



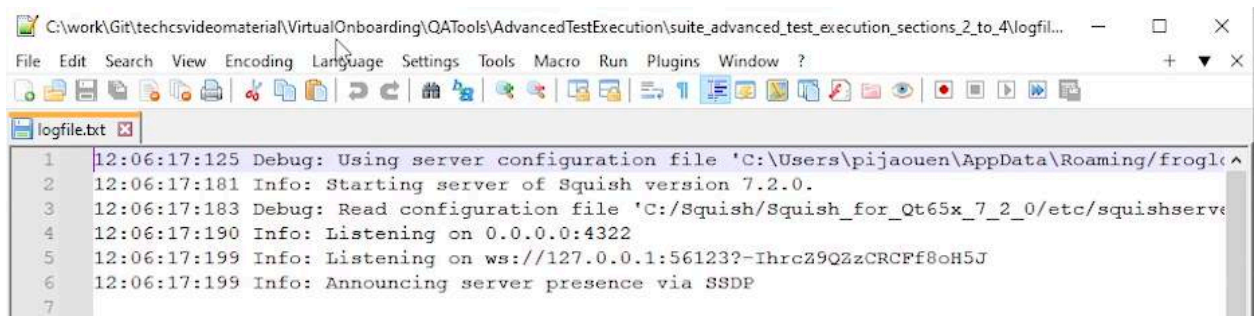
```
Command Prompt - squishserver --verbose --logfile C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_adva...
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishserver --verbose --logfile C:\work\Git\techcsvideomaterial\VirtualOnboarding\
QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\logfile.txt
```

Open a command prompt, navigate to the Squish bin folder, and run the **squishserver** with the **--verbose** and **--logfile** options, specifying where the **logfile.txt** file should be stored. This ensures that all the server activities are logged, providing a thorough record of our testing process.

STEP 02 = Examining the Logfile



```
C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\logfil...
File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?
logfile.txt x
1 12:06:17:125 Debug: Using server configuration file 'C:\Users\pijaouen\AppData\Roaming\frogloa
2 12:06:17:181 Info: Starting server of Squish version 7.2.0.
3 12:06:17:183 Debug: Read configuration file 'C:/Squish/Squish_for_Qt65x_7_2_0/etc/squishserve
4 12:06:17:190 Info: Listening on 0.0.0.0:4322
5 12:06:17:199 Info: Listening on ws://127.0.0.1:56123?-IhrcZ9Q2zCRCFFf8oH5J
6 12:06:17:199 Info: Announcing server presence via SSDP
7
```

After execution, we can see the logfile is created in the test suite folder, and has logged the server's startup details.

STEP 03 = Generating an Excel Report

```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_te...
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --resultdir C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\results --reportgen xls
```

We launch `squishrunner` with the `--testsuite`, `--resultdir`, and `--reportgen` options. We specify that the output should be in Excel `.xls` format and will be saved in the 'results' folder. Note that the file with the log prompted by Squish server has been updated.

STEP 04 = Exploring the Excel Report

START	31/01/2024 12:07	Start 'suite_advanced_test_execution_sections_2_to_4'	Test 'suite_advanced_test_execution_sections_2_to_4' started	
START_TEST_CASE	31/01/2024 12:07	Start 'tst_product'	Test 'tst_product' started	tst_product
PASS	31/01/2024 12:07	C:\work\ Comparison	'2' and '2' are equal	
END_TEST_CASE	31/01/2024 12:07	End 'tst_product'	End of test 'tst_product'	
START_TEST_CASE	31/01/2024 12:07	Start 'tst_sum'	Test 'tst_sum' started	tst_sum
PASS	31/01/2024 12:07	C:\work\ Comparison	'11' and '11' are equal	
END_TEST_CASE	31/01/2024 12:07	End 'tst_sum'	End of test 'tst_sum'	
START_TEST_CASE	31/01/2024 12:08	Start 'tst_subtraction'	Test 'tst_subtraction' started	tst_subtraction
PASS	31/01/2024 12:08	C:\work\ Comparison	'1' and '1' are equal	
END_TEST_CASE	31/01/2024 12:08	End 'tst_subtraction'	End of test 'tst_subtraction'	
END	31/01/2024 12:08	End 'suite_advanced_test_execution_sections_2_to_4'	End of test 'suite_advanced_test_execution_sections_2_to_4'	

Navigating to the results folder and opening the excel spreadsheet reveals a summary similar to the Test Results view in the Squish IDE, offering an organized and concise overview of our test outcomes.

STEP 05 = Generating an HTML Report

```
Command Prompt - squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\AdvancedTestExecution\suite_advanced_te...
Microsoft Windows [Version 10.0.19045.3930]
(c) Microsoft Corporation. All rights reserved.

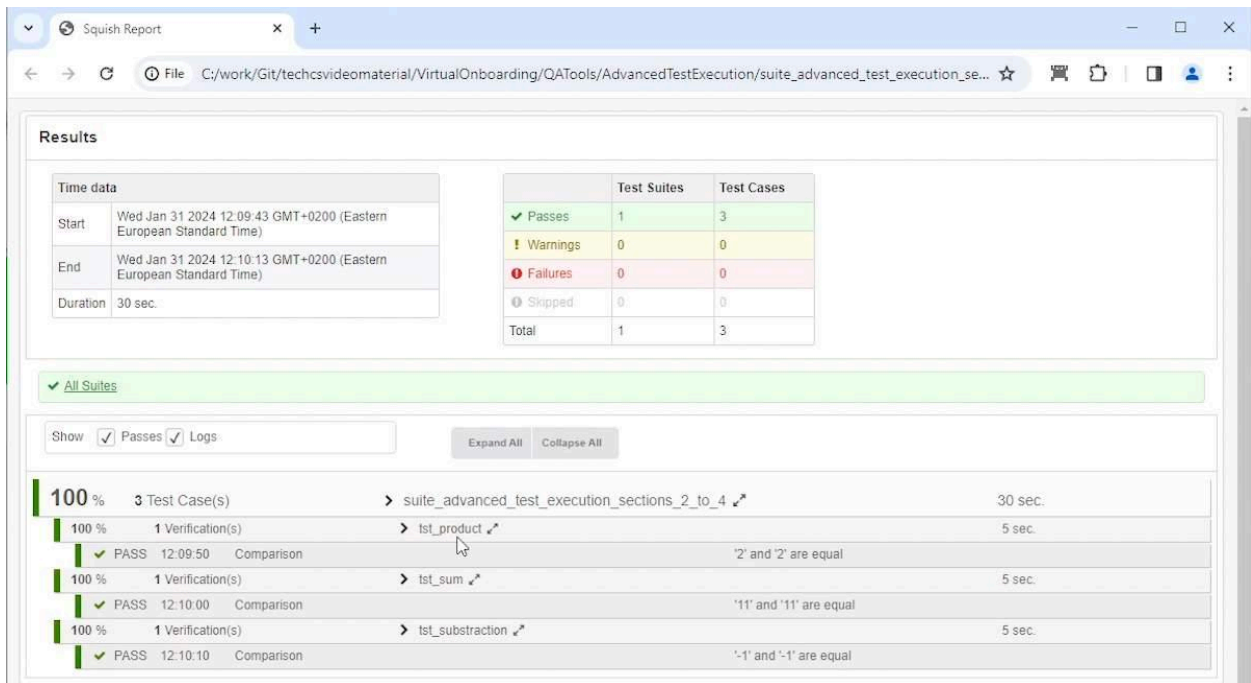
C:\Users\pijaouen>cd %SQUISHDIR%\bin

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\
AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --resultdir C:\work\Git\techcsvideomaterial\VirtualO
nboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\results --reportgen xls

C:\Squish\Squish_for_Qt65x_7_2_0\bin>squishrunner --testsuite C:\work\Git\techcsvideomaterial\VirtualOnboarding\QATools\
AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4 --reportgen html,C:\work\Git\techcsvideomaterial\Vir
tualOnboarding\QATools\AdvancedTestExecution\suite_advanced_test_execution_sections_2_to_4\report
```

If you want a more detailed and interactive summary, use squishrunner with the **html** value in the **--reportgen** option, directing the output to a specific report folder once more.

STEP 06 = Exploring the HTML Report



Open the HTML file, which displays our test results on a webpage. Note that running the command multiple times updates the file with new results, ensuring continuous and comprehensive documentation of our testing.

Summary

Squish Architecture

Performing tests within Squish is built around two key components: the Application Under Test (AUT) and the test script. These operate in separate processes for stability and versatility, especially in remote testing scenarios. The **squishserver** manages communication between the AUT and the test script while the **squishrunner** interprets and executes. This design facilitates testing across multiple platforms and enhances the ease of distributed testing.

Running Tests from the Command Line

Executing tests using Squish from the command line involves initiating the **squishserver** and configuring it with parameters such as: **--host**, **--port**, and **--testsuite** and **--testcase** to execute specific test cases or suites locally or remotely. This method is integral for automating test executions and offers customization options, particularly useful in distributed testing environments such as randomizing the test case.

Tags

Tags in Squish provide a flexible way to organize and select test cases. They can be assigned in the Squish IDE and used with the **--tags** option in **squishrunner** to filter which test cases to execute. This feature allows for more targeted testing, enabling users to run specific groups of tests based on their tags.

Report Creation

Using the **--reportgen** and **--resultdir** options with **squishrunner**, you can generate various types of reports (e.g., HTML, Excel) in specified locations. This functionality is essential for documenting test outcomes and analyzing results in a structured and accessible format.